

LaunchPad Live

Course Landing Page

Evaluation Rubric - Level 3 Final Project | Sessions 11 & 12

Project description

- LaunchPad Live is a responsive single-page website for a small coding school, built with HTML, CSS, Bootstrap, and JavaScript. It introduces the school, displays its courses, roadmap, FAQ, and sign-up form in one organized layout. Course data is managed with JavaScript to calculate launch dates, assign statuses, and filter results in the console. It's a portfolio-style project that combines structure, styling, and functionality in a complete front-end experience.

Instructions

- Design is the student's own. Colors, layout, fonts, and content are the student's choice and are not graded against any sample. Grade only whether the required features and logic are present and working.
- Either approach is accepted where two valid options exist, such as using custom CSS or Bootstrap, and using CSS transitions or keyframe animations.
- Scoring. Award each criterion its points when the work meets the passing criteria. Points are 0/1 only.

Score Summary

Task	Description
Task 1	Project Setup & Layout Organization
Task 2	Bootstrap & Custom CSS Styling
Task 3	Responsive Design with Media Queries
Task 4	JavaScript Data, Functions & Console

Task 1: Project Setup & Layout Organization

Task Details:

1. File Structure: Create a well-organized project folder with index.html at the root and separate /css, /js, and /images directories.
2. File Linking: Link styles.css and app.js correctly within your HTML document.
3. HTML Skeleton: Build the page structure using semantic HTML landmark elements before applying styling or functionality.
4. 8 Required Sections: Build all required sections: Navigation, Hero, About, Roadmap, Courses, Sign Up, FAQ, and Footer.
5. Semantic Tag Usage: Use appropriate semantic HTML elements to organize and structure the content throughout the website.
6. Details/Summary Element: Use the native details and summary elements to create collapsible content for the FAQ section.
7. Layout Organization: Use appropriate CSS layout techniques, such as Flexbox, CSS Grid, or a combination of both, to create a clear, organized, and responsive page layout.

Evaluation Criteria:

Each subtask is evaluated as Accepted or Not Accepted. No partial mark is awarded for a single subtask.

Task	Instructions	Passing Criteria
Project Folder & File Structure	• Create separate /css, /js, and /images folders. • Keep index.html at the project root. • Correctly link styles.css and app.js to the HTML document.	• The required folder structure is present and well organized. • All required files are located in the correct directories. • The HTML file correctly loads the linked CSS and JavaScript files. • The project runs without missing or broken file references.
HTML Skeleton with Landmark Tags	• Build a clear and well-organized page structure using semantic landmark elements. • Ensure each major section includes an appropriate heading.	• The page is divided into clear, meaningful sections that are easy to navigate. • Semantic landmark elements are used to organize the overall page structure. • Each major section includes an appropriate heading. • The document hierarchy makes the content easy to read and understand.
8 Required Sections	• Include all eight required website sections. • Ensure each section is clearly visible on the page.	• The website includes the Navigation Bar, Hero, About, Roadmap, Courses, Sign Up, FAQ, and Footer sections. • Each section contains meaningful content related to its purpose. • Users can easily identify and navigate between the different sections of the website.
Semantic Tag Usage	• Use semantic HTML elements to organize the website content. • Select elements based on their meaning and purpose.	• Semantic elements are used appropriately throughout the page. • HTML elements match the purpose of their content. • The structure improves readability, organization, and accessibility. • Non-semantic elements are only used where appropriate.

Details/Summary for FAQ	• Use the native details and summary elements to create collapsible FAQ content.	• Each FAQ item is implemented using the details and summary elements. • FAQ items can be expanded and collapsed independently. • Questions remain visible while answers are revealed only when expanded.
Flexbox vs Grid Decision	• Use appropriate CSS layout techniques to organize the page content. • Choose the layout approach that best fits each section.	• Appropriate layout techniques are used to organize related content into clear sections. • Content is aligned and spaced consistently throughout the website. • The selected layouts support clear organization, readability, and a responsive user experience.

Task 2: Bootstrap & Custom CSS Styling

Task Details:

1. Bootstrap Features: Use appropriate Bootstrap features to enhance the website's layout and user interface.
2. Custom CSS Styling: Apply custom CSS to create a unique, consistent, and visually appealing website.
3. Color Palette: Create a consistent color palette that enhances the overall visual identity of the website.
4. Buttons & Interactive Elements: Design buttons and interactive elements with a consistent appearance and appropriate visual feedback.
5. Hero Section: Design an engaging hero section that introduces the platform and encourages users to explore the website.
6. Roadmap Section: Present the learning roadmap in a clear and organized layout.
7. Courses Section: Organize and display the available courses in a clear and easy-to-navigate layout.
8. Course Cards: Design visually appealing course cards that clearly present course information and include interactive styling.
9. Visual Consistency: Maintain a consistent visual style across all sections of the website.

Evaluation Criteria:

Each subtask is evaluated as Accepted or Not Accepted. No partial mark is awarded for a single subtask.

Task	Instructions	Passing Criteria
Bootstrap Features	• Use appropriate Bootstrap features throughout the website. • Apply Bootstrap Grid to organize page layouts. • Use Bootstrap components and utilities where they enhance the user interface.	• Bootstrap Grid is used to organize page layouts into responsive rows and columns. • Bootstrap components and utilities are applied appropriately across the website. • Bootstrap features are used consistently across the website to organize the layout and style interface elements.
Custom CSS Styling	• Apply custom CSS to personalize the website. • Use custom styles to enhance the appearance beyond the default Bootstrap styling. • Maintain a consistent styling approach throughout the project.	• Custom CSS is applied across multiple sections of the website. • Custom styles complement the Bootstrap styling without affecting functionality. • Styling is consistent throughout the website and creates a unified appearance.
Color Palette	• Apply a consistent color palette throughout the website. • Use colors to improve readability and highlight important interface elements.	• Colors are used consistently across all website sections. • Color choices improve readability and visual hierarchy. • Important elements such as headings, buttons, badges, or section highlights are visually distinguishable.

Buttons & Interactive Elements.	• Design buttons and interactive elements with a consistent appearance. • Provide visual feedback for user interactions.	• Buttons and interactive elements follow a consistent design throughout the website. • Interactive elements provide visible hover feedback. • Users can easily recognize interactive elements across different sections.
Hero Section.	• Design an engaging Hero section with appropriate content and visual elements. • Ensure the Hero section clearly introduces the website.	• The Hero section includes a clear heading, supporting content, at least one visual element (such as an image or illustration), and at least one call-to-action element. • Visual elements support the website introduction without distracting from the content. • The Hero section creates a clear introduction to the website.
Roadmap Section	• Present the learning roadmap in a clear and organized layout. • Arrange the learning journey in a logical sequence.	• The roadmap displays all required learning steps. • Learning steps are presented in a logical and easy-to-follow order. • The layout clearly separates each step and improves readability.
Courses Section	• Display the available courses in a clear and organized layout. • Present course information in a way that is easy to browse.	• All required courses are displayed. • Each course presents the required information clearly. • Course information is easy to browse and compare.
Course Cards	• Design visually organized course cards with interactive styling. • Maintain a consistent layout for all course cards.	• All course cards follow a consistent layout and styling. • Course cards clearly present the required information. • Interactive effects enhance the user experience without affecting readability or usability.
Visual Consistency	• Maintain a consistent visual style across the website. • Apply consistent typography, spacing, colors, and component styling.	• Typography, spacing, colors, and component styles remain consistent across all sections. • Similar interface elements share a consistent appearance throughout the website. • The overall design appears cohesive and professionally organized.

Task 3: Responsive Design

Task Details:

1. Responsive Strategy: Follow a consistent responsive design approach throughout the website.
2. Desktop Navigation: Ensure the full navigation menu is displayed appropriately on larger screen sizes.
3. Mobile Navigation: Provide a mobile-friendly navigation menu that allows users to access all website sections on smaller screen sizes.
4. Responsive Navigation Menu: Ensure the navigation adapts appropriately when the screen size changes, making all navigation links accessible on smaller devices.
5. Responsive Layout: Organize page content so layouts adapt appropriately across different screen sizes.
6. Custom Media Queries: Use custom media queries where appropriate to enhance the responsive design.

Evaluation Criteria:

Each subtask is evaluated as Accepted or Not Accepted. No partial mark is awarded for a single subtask.

Task	Instructions	Passing Criteria
Responsive Strategy	• Follow a consistent responsive design approach throughout the website. • Ensure the website provides a good viewing experience across different screen sizes	• A consistent responsive design strategy is applied throughout the website. • Content remains readable and well organized across different screen sizes. • The overall user experience is maintained on desktop, tablet, and mobile devices.
Desktop Navigation	• Display the full navigation menu appropriately on larger screen sizes. • Ensure navigation links are clearly visible and easy to access.	• The full navigation menu is visible and well organized on larger screens. • Navigation links are easy to identify and use. • Users can access all main website sections from the desktop navigation.
Mobile Navigation	• Provide a navigation menu suitable for smaller screen sizes. • Ensure users can access all website sections on mobile devices..	• A mobile-friendly navigation menu is available on smaller screens. • Users can easily open the menu and access all navigation links. • The mobile navigation does not interfere with the page content.
Responsive Navigation Menu	• Ensure the navigation adapts appropriately as the screen size changes. • Maintain access to all navigation links across different devices.	• The navigation switches appropriately between desktop and mobile layouts. • All navigation links remain accessible regardless of screen size. • Navigation elements remain clear and usable across different devices.
Responsive Layout	• Organize page layouts to adapt across different screen sizes. • Ensure content remains readable and well structured on different devices..	• Website sections adjust appropriately for desktop, tablet, and mobile screens. • Content remains aligned and easy to read across different screen sizes. • Images, text, and interactive elements fit within the available screen space without unnecessary overflow.

Custom Media Queries	• Use custom media queries where appropriate to improve the responsive design. • Adjust the layout or styling when needed for different screen sizes.	• Custom media queries are used to enhance the responsive experience where appropriate. • Layout, spacing, or typography adjusts smoothly across different screen sizes. • Responsive adjustments improve usability without affecting the overall design consistency.
-----------------------------	--	---

Task 4: JavaScript Data, Functions & Console

Task Details:

1. Date Constants: Declare the required constants used for date calculations.
2. Parallel Arrays: Store course information using parallel arrays for course names, statuses, and launch dates.
3. Days Calculation Function: Create a function that calculates the number of days remaining until a course launch date.
4. Days Label Function: Create a function that converts the calculated number of days into a readable message.
5. Course Filter Function: Create a function that returns courses matching a selected status.
6. Display All Courses: Display all courses with their status and launch information in the browser console.
7. Display Filtered Courses: Display courses that match a selected status in the browser console.
8. Display Upcoming Courses: Display only the courses that have not yet launched in the browser console.

Evaluation Criteria:

Each subtask is evaluated as Accepted or Not Accepted. No partial mark is awarded for a single subtask.

Task	Instructions	Passing Criteria
Constants	• Declare constants to support date calculations throughout the application.	• Constants required for date calculations are declared before they are used. • Constants are used consistently throughout the application instead of hard-coded values.
Parallel Arrays	• Store course names, statuses, and launch dates using parallel arrays. • Keep related course information aligned using the same index.	• Parallel arrays are implemented correctly. • Related course information is stored using matching indexes across all arrays. • Each course's name, status, and launch date remain correctly associated

Days Calculation Function	• Create a function that calculates the remaining time until a course launch date.	• The function accepts a course launch date as input. • The correct number of days is calculated based on the current date. • The returned value can be reused throughout the application..
Days Label Function	• Create a function that converts the calculated number of days into a readable status message. • Cover days left, launching today, and already launched cases.	• The function returns an appropriate message based on the calculated number of days. • Different messages are displayed for upcoming, current, and past launch dates. • Returned messages are clear and suitable for display..
Course Filter Function	• Create a function that filters courses based on their status. • Return only the courses that match the selected status.	• The function accepts a course status as input. • Only matching courses are returned. • The original course data remains unchanged after filtering.
Display All Courses	• Display all available courses using the stored course data. • Include the relevant information for each course in the output.	• Every course is included in the output. • Each course displays its name, status, and launch information. • Output is clearly organized and easy to read.
Display Filtered Courses	• Call filterByStatus using the Open Now status string. • Log the result under a clear heading.	• filterByStatus called with Open Now. • Result logged to the console under a clear heading.
Display Upcoming Courses.	• Check daysLeft() for courses with more than zero days remaining. • Log upcoming course names and days remaining.	• Loop checks daysLeft() greater than zero. • Correct courses logged with name and days remaining under a clear heading.